

Progressive Retrieval Attention for Pretrained Transformers: Sparse Native-K/V Memory for Qwen and Llama-Family Decoders

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 12, 2026

Abstract

Dense long-context attention couples logical memory size to the key/value (K/V) state active in every transformer operation. PRA-HF separates these quantities. It encodes references in model-safe blocks, searches a compact semantic index, and materializes only selected layer-native K/V under a fixed attention budget. Thin Qwen and Llama-family adapters preserve the host model’s rotary, grouped-query, mask, cache, and output-projection semantics; when PRA is disabled, logits, hidden states, and greedy outputs are exactly unchanged.

The central empirical principle is to *route semantic state and attend native positional state*. Contextual hidden states provide the routing index, while selected tokens return as the model’s own post-RoPE keys and native values. A 262,144-parameter router over frozen Qwen3-0.6B reaches any-evidence recall 0.831 at 10% and 0.956 at 20% selected parents. On QASPER, evidence-identity recall at 20% is 0.454 ± 0.007 for Qwen3-0.6B and 0.447 ± 0.051 for the SmolLM2-135M Llama-family control. Oracle HotpotQA replay shows a non-monotone depth effect: the last 14 layers gain 3.641 nats per gold token, whereas all-layer replay loses 0.766. In a bounded implicit-head probe, evidence ranked first and improved gold likelihood by 5.469 nats per token, yet greedy decoding remained wrong. A 98,304-parameter PRA-conditional calibration further improves routed likelihood, but reliable decoded QA gains remain open. The released `pra_hf` package provides the adapters, reference lifecycle, router artifacts, fraction-based selection, accounting, CLI, and reimplementations tests.

1 Introduction

Dense long-context attention couples logical memory size to the K/V participating in each transformer operation. Retaining more history therefore increases both stored state and active attention, even when most of that history is irrelevant to the current token. Prompt truncation breaks the coupling by discarding history. Text retrieval narrows the prompt before inference, but then re-encodes the selected text. PRA instead keeps already computed model state outside the active context and restores only selected native K/V.

Paper 1 showed in controlled models that logical memory and active attention can be separated. Paper 1.5 established the positional contract for sources encoded in bounded blocks. This paper asks whether the same mechanism can be retrofitted into pretrained Hugging Face decoders without changing their native attention semantics. That requirement is strict. Qwen and Llama-family decoders differ in projection details, RoPE helpers, grouped-query layout, masks, cache protocols, and output paths. An adapter can produce plausible text while violating one of these contracts.

Retrofitting also exposes a representational distinction. The state best suited to finding a semantically relevant chunk need not be the state a pretrained attention head expects after selection.

PRA-HF therefore separates a compact *routing index* from the layer-native *detail K/V* payload. The resulting architectural rule, derived empirically below, is: **route semantic state; attend native positional state**.

Figure 1 gives the complete data path before the individual contracts are developed. Publication creates a small search representation and a full native payload; only the selected payload enters active attention.

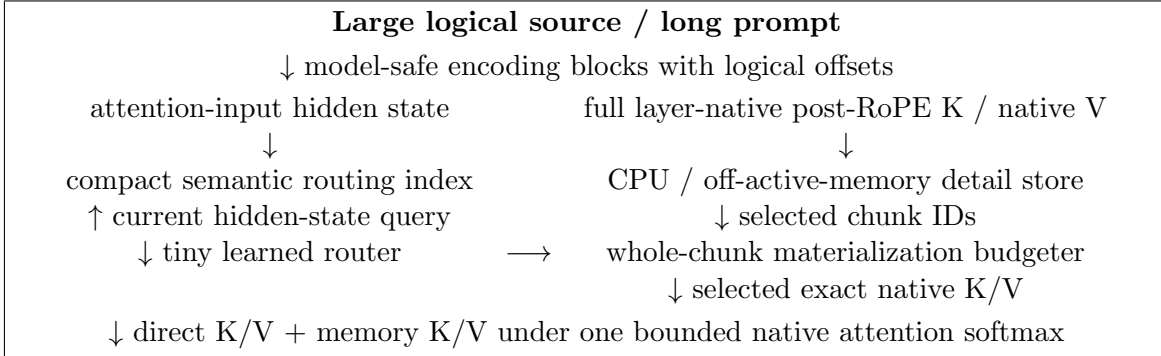


Figure 1: Canonical PRA-HF path. **PRA-HF compresses the search representation, not the selected memory payload.** Routing uses compact semantic state; attention receives the host model’s exact selected positional K/V.

Table 1: Headline findings. The retrofit is exact when disabled, sparse when active, and causally useful at a depth-dependent operating point. Recall definitions and cohorts are stated with each result.

Finding	Result	Why it matters
Exact disabled retrofit	Zero logit and hidden-state difference; identical greedy output	External memory can be added without changing ordinary inference
Semantic routing, native payload	Hidden-state index; selected post-RoPE K and native V	Search features need not replace pretrained attention state
Compact sparse discovery	262,144 parameters; any-evidence recall .831/.956 at 10/20%	A 0.04%-of-model router searches a 0.392%-of-detail index
Depth-dependent causal use	Hotpot oracle $\Delta \log p$: +3.641 last 14; -.766 all 28	More memory exposure is not monotonically better
Bounded-head causal probe	Evidence rank 1; $\Delta \log p = +5.469$; greedy answer wrong	Retrieval, probability change, and decoding are distinct
Portable shared core	QASPER identity recall@20%: Qwen .454 $\pm$.007, SmolLM2 .447 $\pm$.051	Thin family adapters preserve one execution contract
Memory-conditional calibration	98,304 parameters; routed gain +1.846 $\pm$.088 on Hotpot	Small conditional updates beat global readout controls
Reusable artifact	Python API, CLI, saved routers, metrics, tests, wheel	The measured mechanism is independently exercisable

The paper makes five contributions. First, it defines and tests an exact pretrained-model retrofit for bounded native-K/V memory. Second, it identifies routing/payload separation as an

architectural principle rather than a family-specific workaround. Third, it measures sparse discovery by selected-memory fraction and shows that a tiny supervised projection improves frozen semantic geometry. Fourth, it demonstrates that selected native K/V has a causal, non-monotone effect through model depth. Fifth, it releases the shared core, thin adapters, artifacts, and accounting needed to reproduce these claims. The contribution is a working boundary, not a new pretrained model or a solved long-context QA system.

The datasets have different jobs. WikiText-style controls test ordinary language-model integrity and bounded-memory mechanics. QASPER tests one-shot retrieval from a long scientific document. HotpotQA stresses multi-passage discovery and, where direct evidence text helps the checkpoint, provides the cleanest oracle causal probe. It is not a universal pass/fail test for one-shot PRA.

Table 2: Recall metrics answer different questions and are never substituted for one another.

Metric	Meaning
Any-evidence recall	Fraction of examples for which at least one annotated evidence region is selected
Evidence-identity recall	Fraction of all annotated evidence identities recovered
All-evidence recall	Fraction of examples for which every required annotated evidence identity is selected

2 PRA-HF in 60 Seconds

A long source is encoded in blocks that fit the host model. Publication keeps compact semantic gists in a routing index and stores exact layer-native post-RoPE keys and native values as the detail payload, which may remain on CPU. A query searches only the compact index. Selected chunk identities pass through a hard budget, and only their native K/V moves to the accelerator for bounded attention with the direct sequence.

This division answers the most important implementation question. PRA does *not* attend to the gist. The gist is an address label, analogous to a compact catalog entry. Once an address is chosen, attention reads the original native K/V payload. Consequently, routing can use a semantic hidden state without asking a pretrained attention head to consume an unfamiliar representation. Likewise, positional correction is a routing concern only when positional keys are used as gists; the selected attention payload always retains its original native rotary encoding.

The budget distinguishes logical availability from physical activation. A reference may contain thousands of cached tokens, while one operation materializes only B selected tokens plus the direct sequence and a small safety reserve. The mechanism therefore bounds active attention even when the logical source is longer. It does not make storage free: full layer-native detail K/V still occupies CPU memory unless a separate compression or tiering method is added.

Table 3: Four stages of a memory-augmented prediction. Success at one stage does not imply success at the next.

Layer	Question	Present status	Evidence
Transport	Can external native K/V be stored and materialized correctly?	Yes	Exact disabled parity; native GQA K/V; bounded CUDA probes
Discovery	Can relevant memory be selected sparsely?	Strong, domain-dependent	.831 any-evidence recall at 10%; weak cross-domain transfer
Causal use	Does selected memory change the pretrained prediction usefully?	Yes in clean likelihood probes	Hotpot oracle last-half +3.641 nats/token; all-layer replay harmful
Decoded task success	Does the intervention reliably change generated answers?	Not yet	Implicit-head likelihood +5.469; greedy answer remains wrong

3 Minimal PRA Recap

For layer ℓ , PRA stores reference chunks as native $K_c^{(\ell)}, V_c^{(\ell)} \in \mathbb{R}^{H_{kv} \times T_c \times d_h}$ and one or more compact gists $g_c^{(\ell)}$. A query representation $r^{(\ell)}$ scores the gists, and a global budgeter selects whole chunks. Materialized memory precedes the direct K/V sequence under one softmax:

$$\text{Attn}(Q, [K_M; K_D], [V_M; V_D]) = \text{softmax}\left(\frac{Q[K_M; K_D]^\top}{\sqrt{d_h}} + M\right)[V_M; V_D]. \quad (1)$$

This is not a separately normalized cross-attention residual. The native output projection is applied once to the combined result.

Paper 1 established controlled native-K/V routing and materialization. Paper 1.5 separated logical source position from contextual fragmentation. Continuous sources use $p_i = p_{\text{logical start}} + i$, and ordinary post-RoPE keys remain the canonical cache state. The present adapter preserves that contract by capturing each selected HF layer’s actual post-RoPE keys and by assigning source-relative positions across bounded encoding blocks.

4 Exact Retrofitting: Shared Core, Thin Adapters

4.1 Two representations, two jobs

PRA represents a cached chunk as

$$\mathcal{M}_c = (R_c, K_c, V_c), \quad (2)$$

where R_c is a compact routing representation and (K_c, V_c) is the payload used after selection. The objects need not inhabit the same feature space. The canonical HF configuration in this paper sets R_c to one or more gists of the hidden state entering the selected attention block, while K_c is that block’s native post-RoPE key tensor and V_c its native value tensor. Token-level hidden states are transient during publication; only compact gists and detail K/V persist.

This split follows the specialization chain

$$h_{\text{in}} \longrightarrow W_K h_{\text{in}} \longrightarrow \text{RoPE}(W_K h_{\text{in}}). \quad (3)$$

The first representation retains broad contextual information, the second is an attention-address representation, and the third conditions that address on token position. Native token attention needs the third. Chunk retrieval may prefer the first. The routing query therefore uses the final attention-input hidden state and is compared only with hidden-state gists. It is never compared directly with Q_{post} . This is an empirical default for the tested Qwen checkpoint, not a claim that hidden states are universally optimal routers.

4.2 Shared execution core

The shared core divides the former monolithic attention path into explicit stages. Query preparation selects the newest valid state and maps GQA query groups into native K/V routing width. Exact packed-index search returns parent identities. The materialization budget then enforces

$$L_{\text{direct}} + L_{\text{materialized}} + L_{\text{reserve}} \leq L_{\text{native,max}}. \quad (4)$$

`materialize_selected_kv` removes duplicate overlap, preserves row isolation, and moves only retained detail K/V. `combine_local_and_memory_kv` pads variable rows and prepends an additive all-visible memory mask to the family’s existing local causal mask. The controlled model and HF adapters now call this same core.

4.3 Thin HF contract

The abstract adapter exposes six family operations: project Q/K/V, apply native position encoding, normalize tensor layout, combine the native mask, invoke PRA attention, and project the output. Disabled memory takes an even narrower path: call the original attention object unchanged. This design preserves pretrained parameters and avoids checkpoint conversion.

The Qwen adapter has 272 physical lines (248 nonblank, non-comment lines). It introduces no learned parameter. Its family-specific branches distinguish Qwen2-style projections from Qwen3’s Q/K normalization; routing, budgeting, materialization, residency, and metrics remain outside the file. This is a portability measurement, not a code-minimization target.

4.4 Qwen native semantics

4.4.1 Projection and RoPE

The adapter calls the wrapped module’s `q_proj`, `k_proj`, and `v_proj`. Qwen3’s existing `q_norm` and `k_norm` are applied before the installed Transformers `apply_rotary_pos_emb` function. Reference capture saves post-RoPE K and position-independent V . No independent RoPE implementation is introduced.

4.4.2 Grouped-query attention

The tested checkpoint has $H_q = 16$, $H_{kv} = 8$, and $d_h = 128$. Cache objects retain $[B, 8, T, 128]$ K/V. Key-space routing baselines average each pair of query heads sharing one K/V head, producing a vector in the same 8×128 space as a flattened key gist. The canonical hidden-state router instead uses one d_{model} vector and does not alter payload head layout. K/V expansion occurs inside Qwen’s eager attention function, never in persistent PRA storage. A synthetic replay test compares this path with explicit K/V head repetition and matches within 10^{-6} .

4.4.3 HF cache and masks

Prefill and single-token decode both use the wrapped module’s HF `Cache.update` call. PRA memory is prepended after that update, so direct generation history appears once. A decode test with four prompt tokens and two generated tokens observes five direct cache positions at the last invocation, plus four separate memory positions. The original causal mask remains on local/cache positions; selected historical memory receives a visible prefix mask. Memory- disabled generation delegates entirely to the original module.

5 Evaluation Design

Table 4 records the first checkpoint exactly. Tests use synthetic Qwen3 configs offline. The pre-trained run uses Python 3.10.11, PyTorch 2.12.1+CUDA 12.6, Transformers 4.55.4, eager attention, FP16, and an NVIDIA GeForce GTX 950M. The frozen model is not fine-tuned.

Table 4: Pinned Qwen checkpoint and PRA placement.

Checkpoint	Qwen/Qwen3-0.6B
Revision	c1899de289a04d12100db370d81485cdf75e47ca
Parameters / layers	596,049,920 / 28
Query heads / K/V heads / head width	16 / 8 / 128
RoPE base / advertised positions	10 ⁶ / 40,960
Initial PRA placement	layer 27 only
Depth-study instrumentation / selected band	all 28 layers / layers 20–27
Initial gist / routing policy	mean, one gist/chunk, exact top- <i>k</i>
Native limit / direct / selected-memory cap	256 / 128 / 64 tokens
K/V residency	CPU full cache, selected transfer

Every run writes JSON and scalar CSV with the Git SHA, model revision, software versions, device, dtype, layer selection, limits, routing settings, timings, and metrics. The exact executed commit is retained in each artifact rather than inferred from the manuscript build.

6 Exactness and Bounded Execution

6.1 Disabled-path parity

PRA-disabled parity is the hard gate. The baseline is evaluated first; the same in-memory model is then wrapped, leaving every parameter object in place. Table 5 reports exact results for the five-token prompt “The capital of Portugal is” and four-token greedy decode.

Table 5: Original HF model versus memory-disabled PRA wrapper.

Metric	Result
Maximum / mean absolute logit difference	0 / 0
Maximum hidden-state difference	0
Greedy token agreement	100%
Greedy sequences exactly equal	yes
Generation-cache shapes equal	all 28 layers
Finite adapted logits	yes

Single-run prefill times were 0.447 s before wrapping and 0.136 s after wrapping. This order is confounded by first-call warm-up and is not a speed comparison. Exact outputs, rather than latency, are the parity evidence.

6.2 Native-K/V reference transport

An explicit 22-token reference about Lisbon is encoded through the frozen model. The selected layer captures [1, 8, 22, 128] K/V, stores it on CPU, creates a mean gist, and transfers all 22 tokens because the source fits one routing chunk. The physical transfer is 90,112 bytes, exactly $2 \times 22 \times 8 \times 128 \times 2$ bytes for FP16 K and V. The run reports 1.6 ms routing, 0.40 ms materialization orchestration, 0.15 ms selected transfer, and 0.11 ms combined attention. These synchronized phase values are instrumentation checks on one machine, not throughput claims. Cold reference construction took 0.144 s and the warm query took 0.191 s.

The generated continuation contains “Lisbon”, but the unmodified model also knows this fact. Consequently, this observation establishes functional selected-K/V transport and generation, not a causal retrieval benefit. The exact native-replay claim is instead restricted to the synthetic tensor test, where GQA memory plus local K/V equals explicit dense composition.

6.3 Implicit head and bounded long prompt

A repeated prompt tokenizes to 408 logical tokens. The direct cap retains the final 128 tokens; the first 280 become `#_head`. That head is encoded in blocks of at most 64 tokens, with source-relative positions. The direct tail receives positions 280 through 407. Every encoding operation and the final selected-memory operation stays under the 256-token configured native limit; the largest observed source-encoding call is 64 tokens and violations are zero. The final finite-logit query took 0.258 s. This proves bounded execution and offset propagation for the frozen Qwen adapter. It does not show that the router selected the information needed by an arbitrary continuation.

6.4 Transport does not establish discovery

The transport tests intentionally do not establish semantic selection. A diagnostic with one fixed HotpotQA example [6] and one QASPER example [2] shows why. Four conditions use the same frozen checkpoint: question-only truncation, dense text left-truncated to 256 tokens, oracle evidence text-RAG, and PRA with the question direct and the complete source in native-K/V memory. Oracle text-RAG is an upper control because it receives annotated evidence. This two-example matrix diagnoses plumbing; it cannot estimate dataset accuracy.

Table 6: Unrestricted-QA smoke outcomes. Standard normalized token F1 is shown.

Dataset	Condition	F1	Annotated evidence selected
HotpotQA	truncation; dense; oracle text-RAG; PRA	0; 0; 0; 0	PRA: 0%
QASPER	truncation; dense; oracle text-RAG; PRA	0; 0.182; 0; 0.182	PRA: 0%

HotpotQA’s expected answer is “yes”; every condition begins with “No”. QASPER’s expected answer is “no”; dense and PRA generations begin with “No”, while truncation begins with “Yes”. However, PRA routes three tail chunks, none overlapping the annotated evidence, so the QASPER

output cannot be credited to retrieved evidence. Qwen also fails the oracle text-RAG control on both examples. The diagnostic therefore separates a functioning K/V transport path from an unsuitable routing representation; it does not grade the architecture as a QA system.

The selections expose a concrete failure mode. For HotpotQA, evidence lies at token spans 107–137 and 543–588, while routed chunks lie near 1216–1318. For QASPER, evidence lies at 0–317, while selected chunks lie near 4032–4192. Post-RoPE mean-key gists and an unadapted top-layer query systematically prefer irrelevant late-source chunks in these two probes.

7 Semantic Routing, Native Attention

The architectural question is whether native attention state is also a suitable semantic search index. For token j at position p_j , the direct baseline pools

$$g_{\text{post}} = \frac{1}{m} \sum_{j=1}^m R(p_j) K_{\text{pre},j}, \quad (5)$$

which mixes content with different rotary phases. The position-neutral alternative pools $g_{\text{pre}} = m^{-1} \sum_j K_{\text{pre},j}$. A third representation pools the attention-input hidden states. These are hypotheses about ranking only; selected detail memory remains Qwen’s post-RoPE K and native V in every condition.

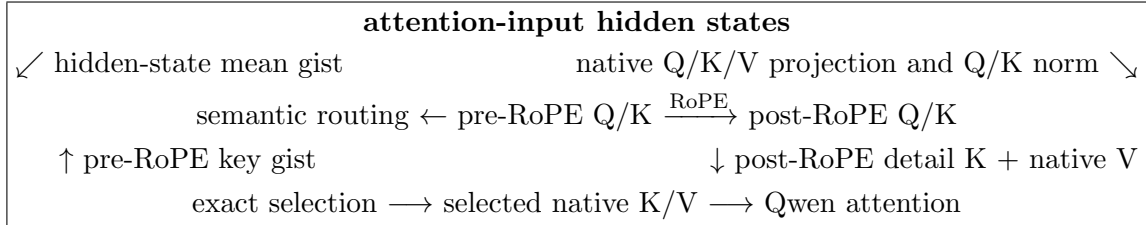


Figure 2: **Route semantic state; attend native positional state.** Pre-RoPE or hidden-state features rank chunks, while the attention path always materializes exact post-RoPE native K/V.

The code makes this independence executable. Routing returns stable *selected chunk identities*; a separate core entry point applies the common budget and materializes their payload. A fixed-selection test supplies the same identity set with ordinary-router and oracle metadata. It obtains bit-identical post-RoPE K , identical V , and exactly identical native attention output. Router choice can change the set, but it cannot silently change what a fixed set means to attention.

7.1 What should represent a chunk?

We first compare three parameter-free summaries on 16 matched examples, eight from HotpotQA and eight from QASPER. All use 32-token parents and the same exact top- k search. Post-RoPE key means mix tokens carrying different rotary phases. Pre-RoPE key means remove position from the index. Attention-input hidden-state means retain broader contextual features. In all three conditions, selection materializes the same post-RoPE native K and native V for a fixed parent identity.

Table 7: Parameter-free routing representations on 16 matched examples. Any-evidence recall measures whether at least one annotated region is selected; score-position r measures correlation with normalized source position.

Routing representation	Any R@3	Any R@8	Any R@16	MRR	Position r
Post-RoPE key mean	0.125	0.250	0.313	0.131	0.652
Pre-RoPE key mean	0.313	0.563	0.750	0.301	0.009
Hidden-state mean	0.438	0.500	0.750	0.276	-0.021

The post-RoPE mean is not merely noisy; it strongly favors late chunks. Removing rotary phase nearly eliminates that bias, and hidden states give the best low-budget result. Centered RoPE is a mechanistic control: it rotates a pooled pre-RoPE gist once at the center of its source span. Eight centered subgists reproduce native token-QK ordering with correlation 0.874, yet their any-evidence recall@3 is only 0.250. The control distinguishes two objectives:

$$\text{native attention geometry} \neq \text{semantic retrieval relevance}. \quad (6)$$

Centering repairs the coordinate frame without creating evidence semantics. We therefore use hidden-state gists for routing and preserve post-RoPE keys only for attention.

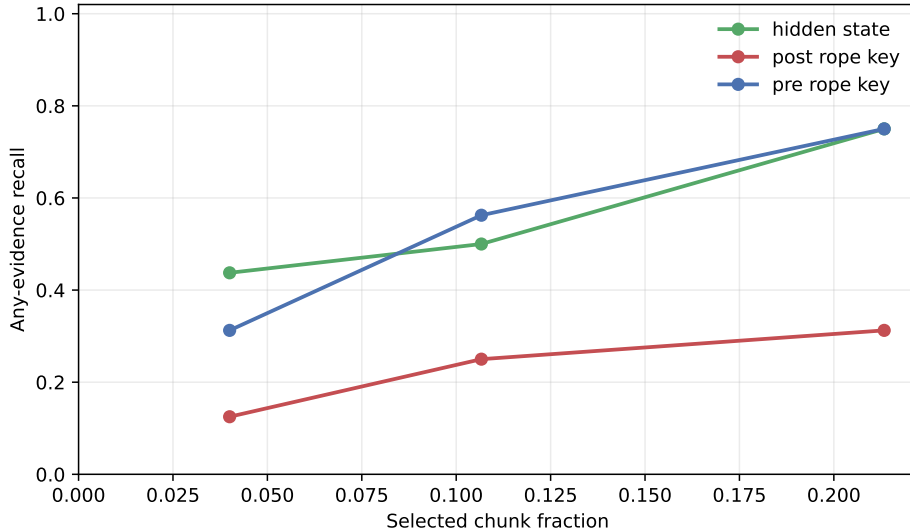


Figure 3: **Semantic hidden-state and pre-RoPE routing dominate post-RoPE key means for evidence selection.** The selected attention payload remains native post-RoPE K/V. These 16-example curves are diagnostic and use a smaller cohort than the learned-router result.

8 Sparse Discovery with a Tiny Learned Router

Parameter-free cosine similarity asks the pretrained hidden space to serve a new retrieval objective without supervision. We instead freeze all 596,049,920 Qwen parameters and learn only two projections,

$$s(q, c) = \text{norm}(W_q h_q)^\top \text{norm}(W_c g_c), \quad (7)$$

where h_q is the last query state and g_c is one mean hidden-state gist. The selected asymmetric 1024-to-128 linear router contains 262,144 parameters, 0.044% of Qwen. Training uses 48 examples, validation uses 16, and the held-out test uses 32, with no QA-identity overlap. Validation selects an exhaustive pairwise margin objective; five independent seeds measure the selected configuration.

Table 8: Held-out sparse ranking. Recall is any-evidence recall; the interval is the 95% half-width over five router seeds.

Router	Any R@3	Any R@8	MRR	Hotpot any R@3	QASPER any R@3
Frozen cosine, last state	0.469	0.719	0.413	0.813	0.125
Learned asymmetric-128	0.631 ± 0.069	0.806	0.498	0.588	0.675
Shuffled-label control	0.125	—	—	—	—

The adapter improves paired any-evidence recall@3 by 0.163 ± 0.069 and removes measurable position bias ($r = -0.026$). The shuffled-label collapse shows that the gain depends on evidence supervision. The learned projection substantially improves the relevance geometry of frozen hidden states while adding only 262,144 parameters. It is not yet a general retriever: Hotpot-to-QASPER and QASPER-to-Hotpot transfer both reach only 0.213 any-evidence recall@3. Wider projections, sampled negatives, and one hard-negative refresh do not improve the validation-selected result; Appendix A reports these controls.

8.1 Recall–sparsity, not fixed rank, defines the operating point

Any-evidence recall@3 is an intentionally severe diagnostic, but it changes meaning as source length changes. For each example with N_i parents, we also select $k_i(f) = \lceil fN_i \rceil$. Historical artifacts support *any-evidence recall*: whether at least one annotated parent is recovered. Complete rankings support the stricter *evidence-identity recall*: the fraction of all mapped evidence parents recovered. We never substitute one definition for the other.

Table 9: Any-evidence recall versus requested parent fraction. The learned row averages five seeds and 160 seed–example evaluations. Other rows are descriptive historical cohorts.

Mechanism	Any R@5%	Any R@10%	Any R@20%	Any R@30%	AUC _{0–.30}
Post-RoPE key mean	0.125	0.125	0.250	0.375	0.190
Pre-RoPE key mean	0.375	0.563	0.938	1.000	0.700
Hidden-state mean	0.438	0.625	0.625	0.813	0.574
Last-state query	0.531	0.719	0.875	0.906	0.724
Learned margin router	0.675	0.831	0.956	1.000	0.835

At requested 10% and 20%, the selected route-once seed recovers 0.356 and 0.442 of annotated evidence identities, while its corresponding any-evidence values are 0.844 and 0.906. These are not the five-seed any-evidence means of 0.831 and 0.956 in Table 9. The seed, cohort, and metric differ. Thus a system can often find one useful passage while still missing other evidence required for a multi-hop answer. This distinction explains why HotpotQA needs a larger realized fraction for 90% any-evidence recall (0.207) than QASPER (0.104).

Scaling must also hold the denominator fixed. In the standalone predecessor, selecting a fixed eight references makes the selected fraction shrink from about 12.5% to 3.1% as memory grows from 64 to 256 units; apparent coverage consequently falls. Selecting approximately 12.5% instead keeps HotpotQA-derived target coverage at 0.756/0.772/0.762 and QASPER-derived coverage at

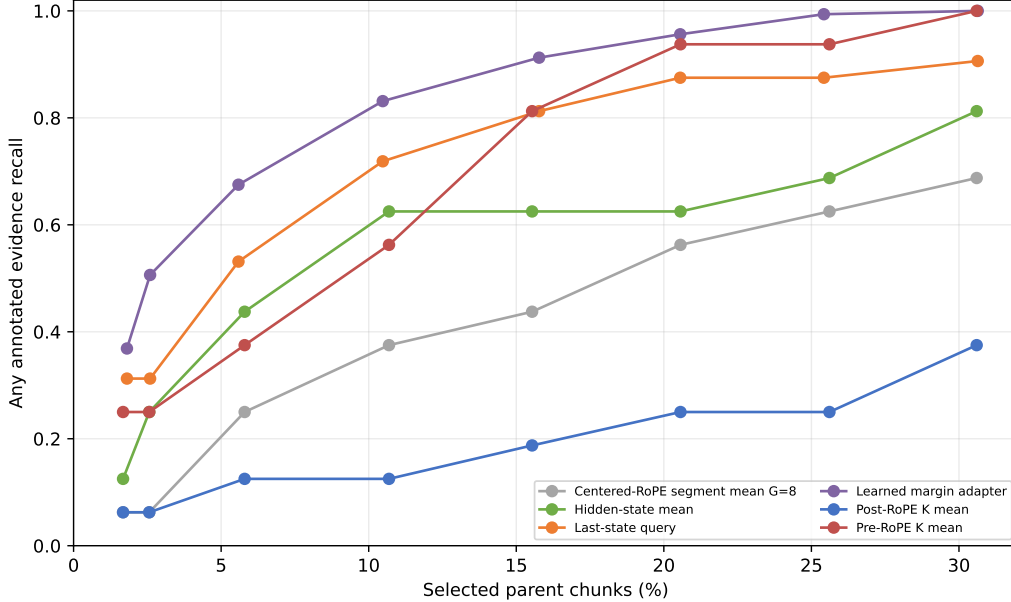


Figure 4: **The tiny learned router moves the recall–sparsity frontier.** Any-evidence recall rises sharply in the 10–20% selected-parent range that fixed-rank evaluation conceals. Cohorts differ for historical baselines, so only the learned result supports uncertainty over seeds.

0.794/0.803/0.800 for 64/128/256 units. Ranking remains stable at a fixed fraction, while physical materialization grows with that fraction. Sparse quality and active-K/V cost must therefore be reported together.

9 Cross-Family Validation

The same artifact and execution interfaces are exercised on Qwen3-0.6B and the public SmolLM2-135M Llama-family checkpoint. Both routers use frozen last-layer attention-input states, one mean per 32-token parent, 128-dimensional asymmetric projections, 512 optimization steps, QASPER training, and seeds 11, 23, 37, 53, and 71. Table 10 reports evidence-identity recall, not the any-evidence metric above.

Table 10: Five-seed QASPER test evidence-identity recall. This stricter metric is the fraction of annotated identities recovered, not the any-evidence metric in Table 9. Values are mean $\pm$ sample standard deviation where shown.

Family	Identity R@5%	Identity R@10%	Identity R@20%	Identity R@30%	AUC _{0:30}
Qwen3-0.6B	0.293	0.362	0.454 $\pm$ 0.007	0.532	0.396
SmolLM2-135M	0.243	0.334	0.447 $\pm$ 0.051	0.532	0.371

The means at 20–30% are close, but Qwen is more stable and has the stronger sparse AUC. This supports software and tensor-contract portability, not universal transfer of routing quality. The official Meta Llama 3.2-1B checkpoint remains an unmeasured target because its manually gated repository returned HTTP 401 in the available environment. No SmolLM2 number is presented as a Meta Llama result.

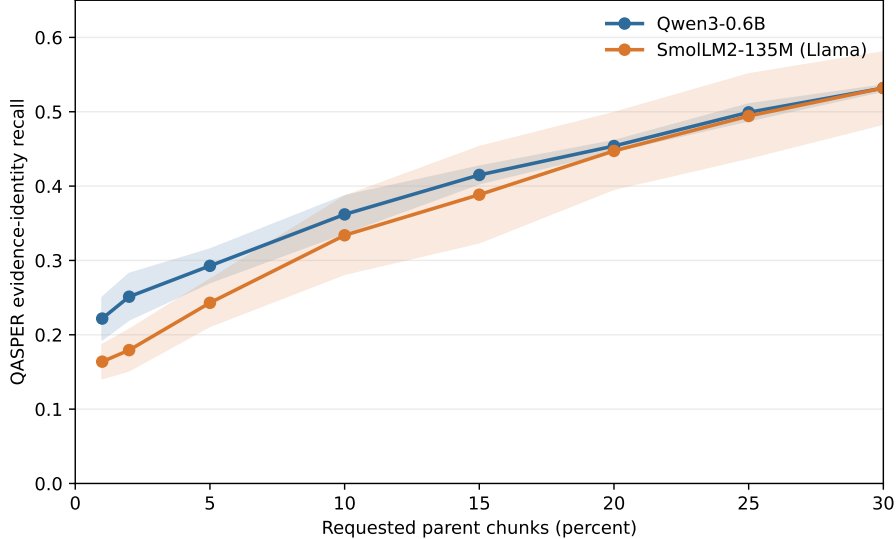


Figure 5: **The shared PRA-HF core ports through thin family adapters.** Matched QASPER evidence-identity recall is similar for Qwen3-0.6B and the SmolLM2-135M Llama-family control, although neither reaches 0.80 before exhaustive selection.

10 Causal Use Is Depth Dependent

Retrieval recall asks whether annotated evidence was selected. Causal evaluation asks a different question: does the frozen decoder assign more probability to the gold answer after that evidence becomes native K/V? We intervene at both the selection and consumption boundaries. The no-memory condition supplies only the question. Oracle conditions force every 32-token parent intersecting an annotated evidence span. The direct-text control places the same evidence in the ordinary prompt. Four deterministic examples from each dataset make this a mechanism probe rather than an accuracy estimate.

Forced identities enter `prepare_selected_memory`, exactly where ordinary semantic routing ends. Both routes then share thresholding, whole-parent budgeting, native post-RoPE K/V, CPU-to-GPU transfer, GQA layout, source positions, additive masks, and Qwen’s eager attention kernel. A selected parent identity is fixed across layers, but its tensor is not: layer ℓ receives $K_c^{(\ell)}, V_c^{(\ell)}$ captured from that layer’s own Qwen projections and RoPE path. The 640-token bound reserves 128 direct tokens and permits up to 512 memory tokens per layer. Every oracle parent survives; no budget or native-limit violation occurs, and each reference-encoding call is at most 128 tokens. “Active K/V” is the sum of materialized physical tokens over consuming layers, not the number of distinct source tokens.

Generation-only F1/EM is too coarse for this small frozen checkpoint. The primary paired outcome is therefore teacher-forced mean gold-token log-probability,

$$\Delta_{c,i} = \frac{1}{|y_i|} \log p(y_i | q_i, c) - \frac{1}{|y_i|} \log p(y_i | q_i, \emptyset), \quad (8)$$

with first-token probability/rank, generated text, attention mass, and answer-position hidden state deltas retained in per-example traces. Four deterministic examples from each dataset are used. This is a mechanism probe, not an accuracy estimate.

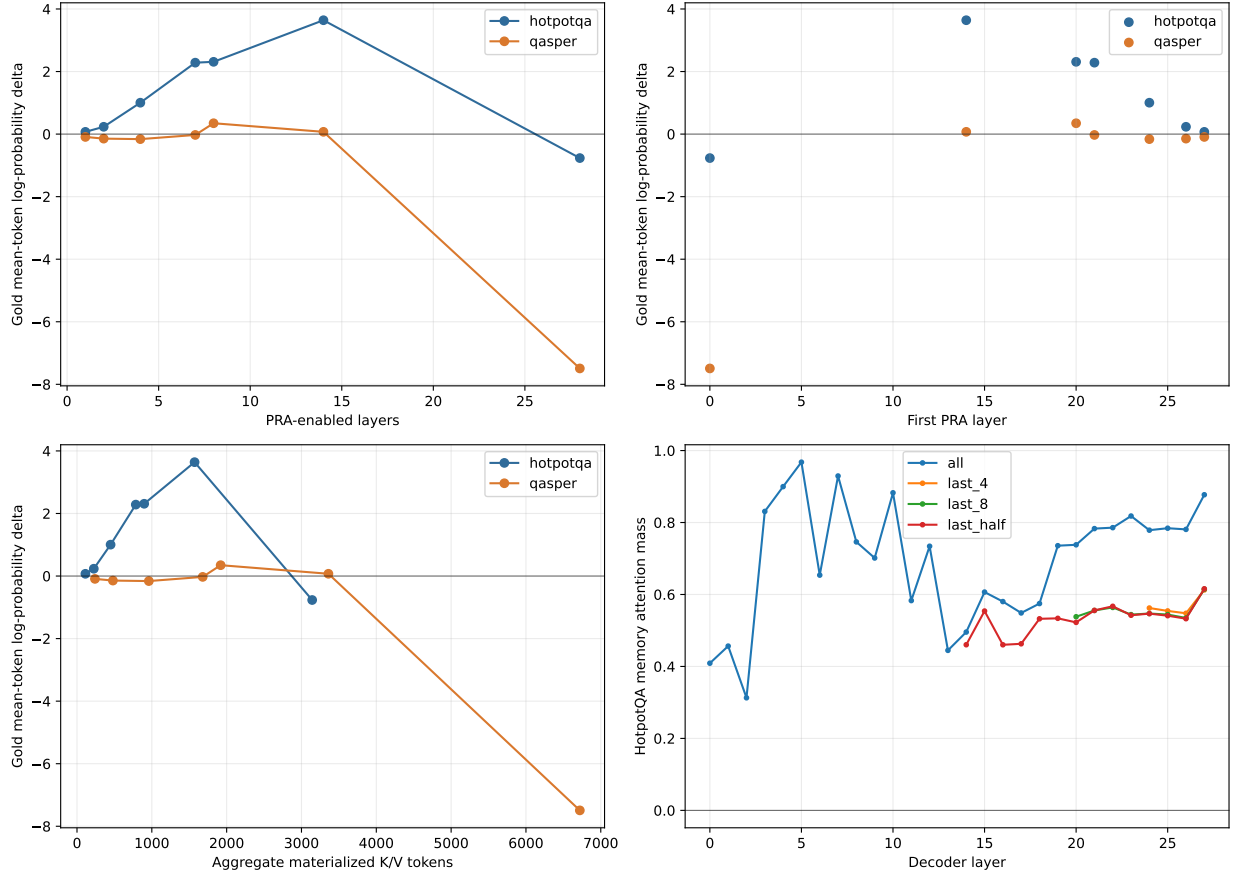


Figure 6: **Retrieved native K/V has a depth-dependent causal effect.** A contiguous late band approaches the direct-text benefit on HotpotQA, whereas all-layer sparse injection is harmful. The lower panels show that additional exposure also increases materialization cost.

Table 11: Oracle consumption-depth sweep. $\Delta \log p$ is paired mean gold-token log-probability relative to no memory. K/V is aggregate physical materialization across active layers. The key comparison is non-monotone: a late band helps on HotpotQA, whereas all-layer injection harms both datasets.

Dataset	Condition	Layers	$\Delta \log p$	F1	K/V tokens	TTFT	Generation
HotpotQA	none	0	.000	.056	0	.140	.737
HotpotQA	oracle, last 1	1	+.071	.056	112	.143	.733
HotpotQA	oracle, last 4	4	+1.004	.056	449	.154	.813
HotpotQA	oracle, last 8	8	+2.311	.056	898	.169	.877
HotpotQA	oracle, last half	14	+3.641	.056	1,572	.193	1.013
HotpotQA	oracle, all	28	-.766	.000	3,143	.245	1.382
HotpotQA	direct evidence text	0	+4.635	.125	0	.198	.986
QASPER	none	0	.000	.125	0	.135	.781
QASPER	oracle, last 1	1	-.091	.125	240	.137	.785
QASPER	oracle, last 4	4	-.161	.125	960	.149	.844
QASPER	oracle, last 8	8	+.347	.125	1,920	.163	.899
QASPER	oracle, last half	14	+.072	.100	3,360	.188	1.081
QASPER	oracle, all	28	-7.492	.000	6,720	.249	1.488
QASPER	direct evidence text	0	-1.791	.000	0	.416	1.090

HotpotQA supports the integration-depth hypothesis within a late band. The last-one, last-four, last-eight, and last-half interventions move mean log-probability by +0.071, +1.004, +2.311, and +3.641 nats/token. The last eight recover 49.9% of the direct-text effect while using 8/28 layers; the last half recover 78.6% at 1.75 times the last-eight materialization. Generated F1 does not change. The evidence therefore establishes causal likelihood use, not decoded QA success.

This depth dependence is plausible because a residual transformer accumulates context-conditioned updates,

$$h^{(\ell+1)} = h^{(\ell)} + \Delta h^{(\ell)}. \quad (9)$$

Memory introduced too late has few opportunities to affect that trajectory. Memory introduced at every layer can interfere before the local question representation has formed. The measured late band lies between those extremes, but the small probe does not establish a universal theory of transformer hierarchy.

QASPER separates perturbation from interpretable utility. It turns positive at the last eight layers (+0.347) but is only +0.072 at the last half, while direct evidence text is negative (−1.791). That control failure prevents using this subset to grade PRA transport. Attention mass and hidden-state perturbations are retained in the artifact, but neither alone is evidence that a perturbation is useful.

10.1 Placement is not interchangeable

The depth result justifies one placement follow-up. Table 12 fixes either four or seven consuming layers and leaves oracle parent identities, positions, prompts, and per-layer payload sizes unchanged.

Table 12: Oracle placement at fixed memory identities. Equal-count rows have equal aggregate K/V cost, isolating where memory enters the residual trajectory.

Placement	Layers	First layer	$\Delta \log p$	F1
Early contiguous	4	0	-10.573	.000
Middle contiguous	4	12	-.556	.090
Evenly spaced	4	0	-10.398	.000
Late contiguous	4	24	+.421	.090
Every fourth	7	0	-6.090	.000
Late contiguous quarter	7	21	+1.128	.090

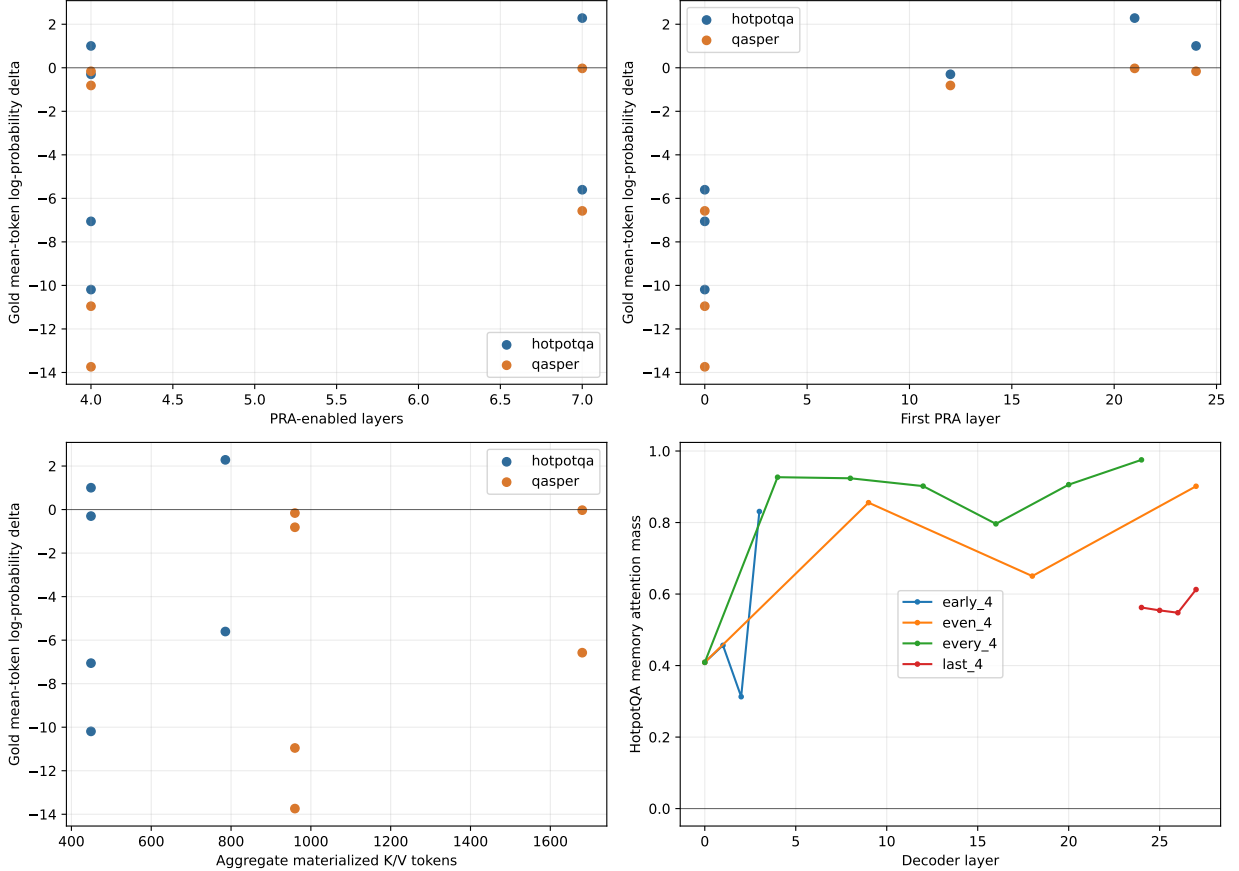


Figure 7: **A contiguous late band is not interchangeable with earlier or sparse placement.** At matched parent identities and per-layer payload, earlier schedules reverse the likelihood effect despite equal K/V cost at equal layer counts.

This rejects both proposed shortcuts. Earlier insertion does not help by leaving more downstream depth, and every-fourth insertion does not approximate dense late consumption. The likely mechanism is a layer-specific contextualization/topology mismatch: independently encoded reference states are least compatible with early residual computation, whereas a contiguous upper band can repeatedly integrate them after the local question has become semantically formed. This interpretation is mechanistic, not yet a general theorem about decoder depth. All-layer replay is especially informative: more attention opportunities can make the answer worse. The operating

point is therefore a band, not “as many PRA layers as possible.”

10.2 Route once, consume with layer-native payloads

The routed follow-up scores the question once at layer 27, selects three parent IDs, and reuses those identities across either the last 8 or last 14 layers. Every layer resolves the ID to its own post-RoPE K/V; no tensor crosses layer boundaries. Table 13 reports the complete pre-top- k evidence-identity ranking and the aggregate consumption cost.

Table 13: Combined route-once result over eight examples. Recall is evidence-identity recall, the fraction of mapped evidence-parent identities recovered; f_{80} and f_{90} are the first measured parent fractions reaching those levels.

Router	Consumption	ID R@5%	ID R@10%	ID R@20%	ID R@30%	f_{80}	f_{90}	$\Delta \log p$	F1	K/V
Learned margin	last 8	.264	.403	.521	.639	.502	1.000	+ .227	.090	768
Learned margin	last half	.264	.403	.521	.639	.502	1.000	+ .374	.028	1,344
Raw hidden-state cosine	last 8	.092	.224	.414	.573	1.000	1.000	+ .150	.090	768
Raw hidden-state cosine	last half	.092	.224	.414	.573	1.000	1.000	+ .568	.090	1,344

The learned router gives the stronger sparse ranking and reaches evidence in 5/8 selected top-3 sets. Last-eight consumption preserves the no-memory F1 and moves likelihood by +0.227; last-half consumption moves it by +0.374 but lowers F1 to .028. Raw cosine has weaker sparse recall yet a larger last-half likelihood shift. This is not a contradiction: a selected non-annotated parent can still change the answer distribution, and a likelihood movement need not cross the greedy decoding boundary. The learned last-eight condition is the conservative quality/cost operating point. It materializes 768 K/V tokens across layers, increases TTFT from .141 to .167 seconds and generation from .759 to .907 seconds, and adds one .149-second query encoding plus .021-second exact route. Its layer-27 learned index averages 45,184 bytes; storing projected gists for all 28 instrumented layers averages 1.27 MB, versus 322.7 MB of resident detail K/V. These eager single-example timings characterize this implementation only.

10.3 Retrieval, causal influence, and decoding are distinct

The earlier single-layer `#_head` run established bounded displacement but generated no correct answer. We repeat that probe with the learned router at layer 27 and the selected late-eight consumption band. The 416-token logical prompt displaces 288 tokens, retains a 128-token direct tail with positions 288–415, and indexes nine 32-token parents. The router ranks the evidence parent first and selects it with two neighboring parents. Each layer 20–27 then consumes its own 96-token native K/V payload.

Table 14: Bounded implicit-head confirmation. The router ranks the evidence first and selected memory strongly improves the gold distribution, but the greedy answer remains wrong. Rank is the gold first-token vocabulary rank; K/V is aggregate physical materialization.

Condition	Evidence rank	$\Delta \log p$	First-token rank	K/V	F1	Viol.
No memory	—	.000	94,106	0	.000	0
Learned route, last 8	1	+5.469	1,351	768	.000	0

The intervention raises the two-token answer’s mean log-probability from -10.924 to -5.455 and its first-token probability from 3.79×10^{-9} to 1.95×10^{-5} . Greedy output still omits “cobalt.”

The maximum native operation remains 128 tokens and no violation occurs. Thus one bounded run demonstrates successful discovery, layer-native transport, and strong causal likelihood use together. It also shows why they must be measured separately from decoded task success: a five-order-of-magnitude first-token probability increase need not cross the greedy decision boundary.

10.4 A tiny residual adapter aligns memory with the frozen decoder

The preceding experiments expose a narrower problem than failed retrieval. Correct late-layer memory can improve the gold distribution, yet the same memory effect is not necessarily aligned with the trajectory expected by a decoder pretrained without external K/V. We therefore freeze the transformer, LM head, and router and adapt only the difference caused by memory. Let y_l^{mem} be the native attention output with selected memory, let y_l^{local} be a second attention evaluation with only the prompt K/V, and define the counterfactual memory effect $d_l = y_l^{\text{mem}} - y_l^{\text{local}}$. A scalar gate uses

$$y_l = y_l^{\text{local}} + \alpha_l d_l. \quad (10)$$

The residual adapter instead retains frozen PRA at initialization and learns a correction:

$$y_l = y_l^{\text{local}} + d_l + W_{\text{up},l} \text{SiLU}(W_{\text{down},l}[h_l; d_l] + b_l). \quad (11)$$

W_{up} is initialized to zero, so the initial function is exactly frozen PRA. The down projection maps $2D$ features to a bottleneck of 16, 32, or 64; the up projection maps back to D . One adapter is installed at each of Qwen’s final four layers. When PRA is disabled the original attention module is called directly, bypassing both the counterfactual attention and the adapter.

We train on eight HotpotQA identities for 24 oracle-memory updates at learning rate 10^{-3} . The base model, LM head, and learned router remain frozen. Evaluation uses four identity-disjoint HotpotQA examples and four QASPER examples, under oracle and learned-router selection. The data cohort is fixed while bottleneck initialization and example order vary over seeds 11, 23, 37, 53, and 71. Table 15 reports means across those five optimization seeds; the fixed baseline is deterministic on the same examples.

Table 15: Frozen late-four-layer memory calibration. $\Delta \log p$ is mean gold-token log-probability relative to no memory. F1 uses routed greedy output. Parameter percentages use the 596.0M-parameter base model.

Method	Params	% base	HotpotQA $\Delta \log p$		QASPER $\Delta \log p$		Routed F1	
			Oracle	Routed	Oracle	Routed	HotpotQA	QASPER
Frozen PRA	0	.000	+1.004	+.110	-.161	-.456	.056	.125
Scalar gate	1	<.001	+1.206	-.187	+.114	-.663	.056	.063
Residual 16	196,672	.033	+2.867	+1.176	+.002	-.161	.043	.000
Residual 32	393,344	.066	+3.088	+1.210	+1.022	+.837	.063	.013
Residual 64	786,688	.132	+2.697	+.896	+1.079	+.853	.072	.017

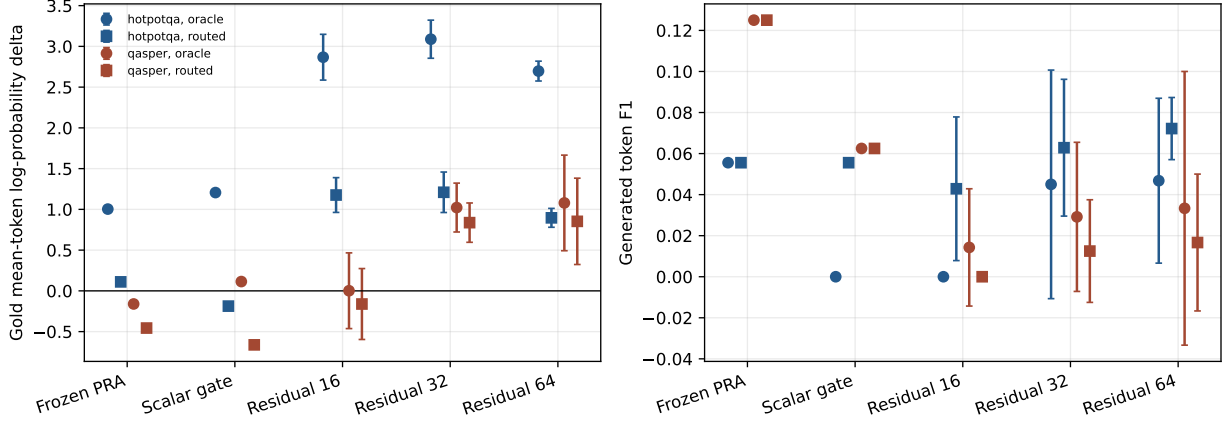


Figure 8: Five-seed memory-calibration result. Error bars are population standard deviations over optimization seeds. The residual adapters improve gold likelihood, but those gains do not translate into commensurate greedy token F1.

Width 32 is the smallest robust operating point. Its routed HotpotQA gain is 1.210 ± 0.248 nats per token, compared with $+110$ for frozen PRA; its routed QASPER gain is $+837 \pm .241$, compared with $-.456$. Each paired difference favors width 32 in all five seeds. With only five pairs, however, the smallest exact two-sided sign-flip result is $p = .0625$, and the examples are not independent dataset replications. On HotpotQA, width 32 improves the first-token margin by 3.063 logits and lowers the first-token rank by 770 positions relative to no memory. On QASPER it improves the margin by 1.280 logits while worsening rank by 8.05 positions, another warning that sequence likelihood, first-token ordering, and greedy decoding are distinct outcomes.

The adapter does not solve QA. EM remains zero, HotpotQA F1 changes only from .056 to .063, and QASPER F1 falls from .125 to .013. The result supports the narrower interpretation that useful PRA-conditioned state already exists and a 0.066%-of-model readout can align its gold-sequence likelihood with the frozen decoder. It does not show that this tiny cohort teaches robust answer generation. Width 32 occupies 1.50 MiB in FP32. Routed teacher-forced latency rises from .163 to .172 seconds on HotpotQA and from .160 to .162 seconds on QASPER; generation timing is noisy and shows no consistent width trend. PRA-off gold scores remain exactly equal in every run, active K/V materialization is unchanged, and there are no native-limit violations.

10.5 Conditional late-band LoRA is more parameter-efficient

The depth sweep locates a useful late transformer band, so a compatibility update need not touch the entire checkpoint. We restrict LoRA to the native attention output projection in Qwen’s final four PRA-enabled layers. If $z_l \in \mathbb{R}^{B \times T \times D_a}$ is the flattened multi-head attention result before the native output projection, the active memory branch is

$$y_l = W_{O,l}z_l + \frac{\alpha}{r}B_lA_lz_l, \quad A_l \in \mathbb{R}^{r \times D_a}, \quad B_l \in \mathbb{R}^{D \times r}. \quad (12)$$

The pretrained W_O is frozen. A_l uses Kaiming initialization and B_l starts at zero, so Equation 12 initially equals frozen PRA. The correction executes only after selected memory has been materialized; no-memory and disabled-PRA calls delegate to the original attention module. This distinction is stricter than applying ordinary LoRA globally: the new parameters calibrate the checkpoint’s

response to external native K/V without changing its local attention function. Qwen3’s grouped-query attention produces a $D_a = 2048$ concatenated head vector before a rectangular $2048 \rightarrow 1024$ output projection. Thus each layer uses $r(2048+1024)$ parameters rather than the square-projection count; over four layers, ranks 2, 4, and 8 contain 24,576, 49,152, and 98,304 parameters.

The protocol matches the residual study: eight HotpotQA training identities, 24 oracle-memory updates, four held-out identities per dataset, and optimization seeds 11, 23, 37, 53, and 71. The base model, LM head, router, and all inactive calibrators remain frozen. We set $\alpha = r$ and dropout to zero. Every update also evaluates the same prompt with PRA disabled and penalizes any logit drift; the control error is exactly zero because the conditional branch is absent. At test time we measure no memory, routed PRA, oracle PRA, and direct evidence text.

Table 16: PRA-conditional late-band calibration. Values are mean $\Delta \log p$ relative to no memory; uncertainty is the population standard deviation across five optimization seeds. F1 is routed greedy token F1. The residual-32 row is the matched larger calibrator.

Method	Params	% base	HotpotQA $\Delta \log p$		QASPER $\Delta \log p$		Routed F1	
			Oracle	Routed	Oracle	Routed	HotpotQA	QASPER
Frozen PRA	0	.0000	+1.004	+.110	-.161	-.456	.056	.125
Residual 32	393,344	.0660	+3.088 $\pm$.234	+1.210 $\pm$.248	+1.022 $\pm$.300	+.837 $\pm$.241	.063	.013
Output LoRA $r = 2$	24,576	.0041	+1.794 $\pm$.111	+.889 $\pm$.111	+.695 $\pm$.081	+.505 $\pm$.082	.056	.125
Output LoRA $r = 4$	49,152	.0082	+2.248 $\pm$.100	+1.347 $\pm$.090	+1.175 $\pm$.057	+1.021 $\pm$.053	.057	.125
Output LoRA $r = 8$	98,304	.0165	+2.775 $\pm$.092	+1.846 $\pm$.088	+1.655 $\pm$.055	+1.487 $\pm$.062	.071	.063

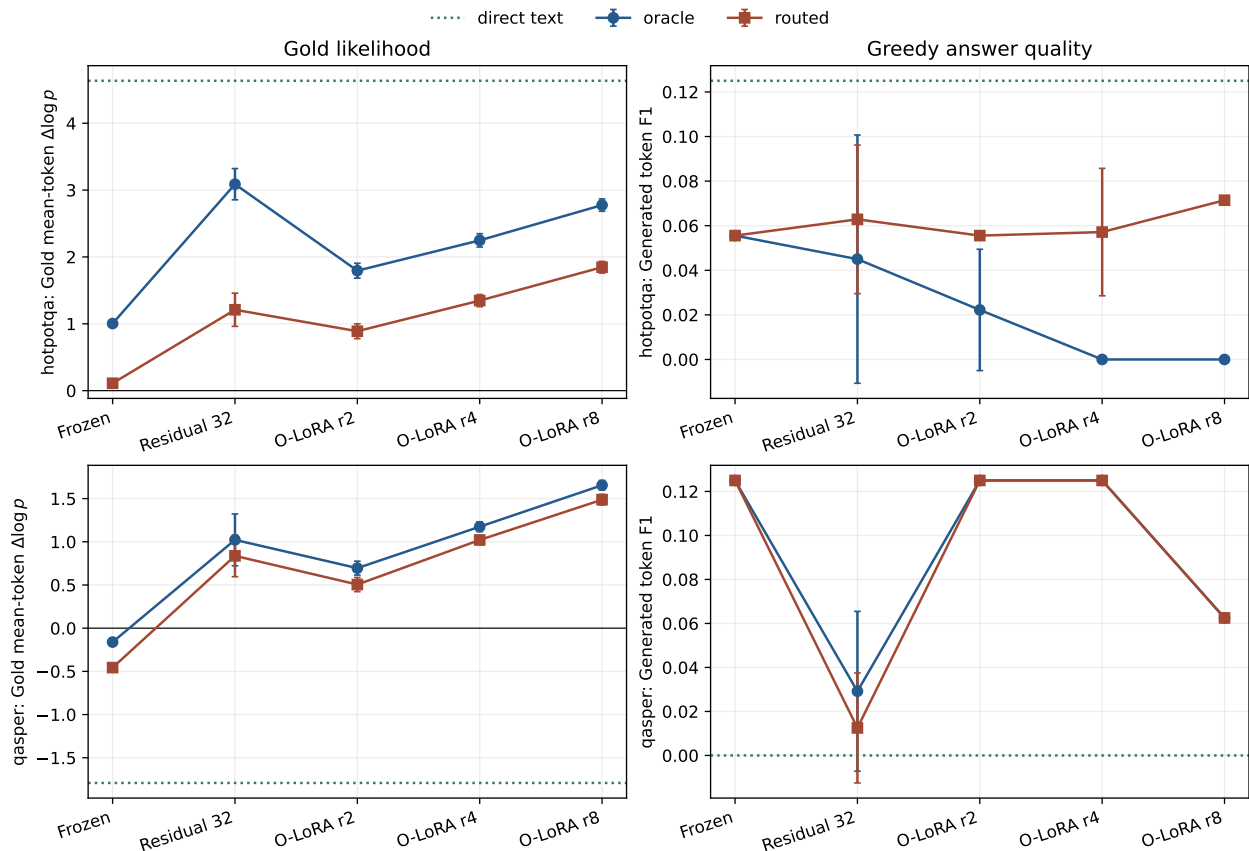


Figure 9: Five-seed late-band comparison. Output-projection LoRA improves routed and oracle gold likelihood monotonically with rank in this sweep. Greedy answer quality does not follow the same trend; QASPER direct text is itself negative on this four-example cohort and is not evidence of QA utility.

Rank 8 raises routed likelihood over residual-32 by .637 nats/token on HotpotQA and .650 on QASPER while using one quarter of its trainable parameters. The paired difference has the same direction in all five seeds; as before, five pairs permit no exact two-sided sign-flip result smaller than $p = .0625$. On HotpotQA, rank 8 lowers mean first-token rank by 633 positions and raises its margin by 4.627 logits relative to no memory. Residual-32 lowers rank further, by 770, but has a smaller margin gain of 3.063 and lower sequence likelihood. These metrics are related but not interchangeable.

The result remains a calibration result rather than reliable generation. Routed HotpotQA EM is zero and F1 is .071; QASPER EM is zero and F1 falls to .063. Rank 8 occupies 384 KiB in FP32. Mean routed teacher-forced time is .168 seconds on HotpotQA, only .003 seconds above frozen PRA, and .158 seconds on QASPER. Active K/V remains 384 tokens across the four consuming layers, disabled-PRA logits are bit-exact in every run, and no native-limit violation occurs. Because the output-only sweep already separates likelihood calibration from decoding, broader Q/O or MLP updates are deferred until a larger calibration/validation/test cohort can decide whether their extra degrees of freedom improve answer generation rather than fit eight identities.

10.6 A staged adaptation sequence rejects global readout tuning

We close the calibration study with the predefined sequence: decoding diagnostics, scalar gate, conditional residual adapter, conditional late-band LoRA, and only then a global LM-head control. Every trained stage uses the same eight HotpotQA identities, 24 oracle-memory updates, four held-out HotpotQA identities, and seeds 11, 23, 37, 53, and 71. Evaluation always includes no memory, routed PRA, oracle PRA, and direct evidence text. The direct-text HotpotQA control is positive at +4.635 nats/token and F1 .125, so this cohort can distinguish model failure from an unanswerable prompt. QASPER remains a supplementary sensitivity result because its direct-text control is negative on the four selected cases.

The last control changes the global vocabulary readout. Qwen ties its native $151,936 \times 1024$ LM head to token embeddings, so “head only” would otherwise update the input embedding table too. We explicitly clone and untie the output matrix while freezing the embedding. We test rank-8 head LoRA, the full untied head, and final RMS normalization plus that head. The two full-matrix variants use state-free SGD because AdamW and Adafactor optimizer state exceed the 4-GiB evaluation GPU; head LoRA uses AdamW. All optimize oracle-memory likelihood plus KL distillation from frozen no-PRA answer-position distributions. Ordinary behavior is measured on 1,024 fixed WikiText-2 validation tokens, not inferred from QA alone.

Table 17: Final HotpotQA adaptation sequence. $\Delta \log p$, rank, and margin are for routed PRA relative to each method’s own no-memory baseline; uncertainty is population standard deviation over five optimization seeds. Weight memory excludes optimizer state. “Exact” means a structural PRA-off bypass.

Method	Params	Weight MiB	PRA-off	$\Delta \log p$	Rank	Δmargin	F1/EM	Time (s)
Frozen PRA	0	0	yes	+ .110	441	+ .104	.056/.000	.163
Scalar gate	1	<.001	yes	$-.187 \pm .007$	160	+ .733	.056/.000	.173
Residual 32	393,344	1.50	yes	$+1.210 \pm .248$	117	+3.063	.063/.000	.172
Late output LoRA $r = 8$	98,304	.375	yes	$+1.846 \pm .088$	254	+4.627	.071/.000	.168
LM-head LoRA $r = 8$	1,223,680	4.67	no	$+ .104 \pm .009$	445	+ .096	.056/.000	.171
Full LM head	155,582,464	296.75	no	$+ .090 \pm .026$	444	+ .075	.056/.000	.164
Final norm + head	155,583,488	296.75	no	$+ .090 \pm .026$	444	+ .074	.056/.000	.163

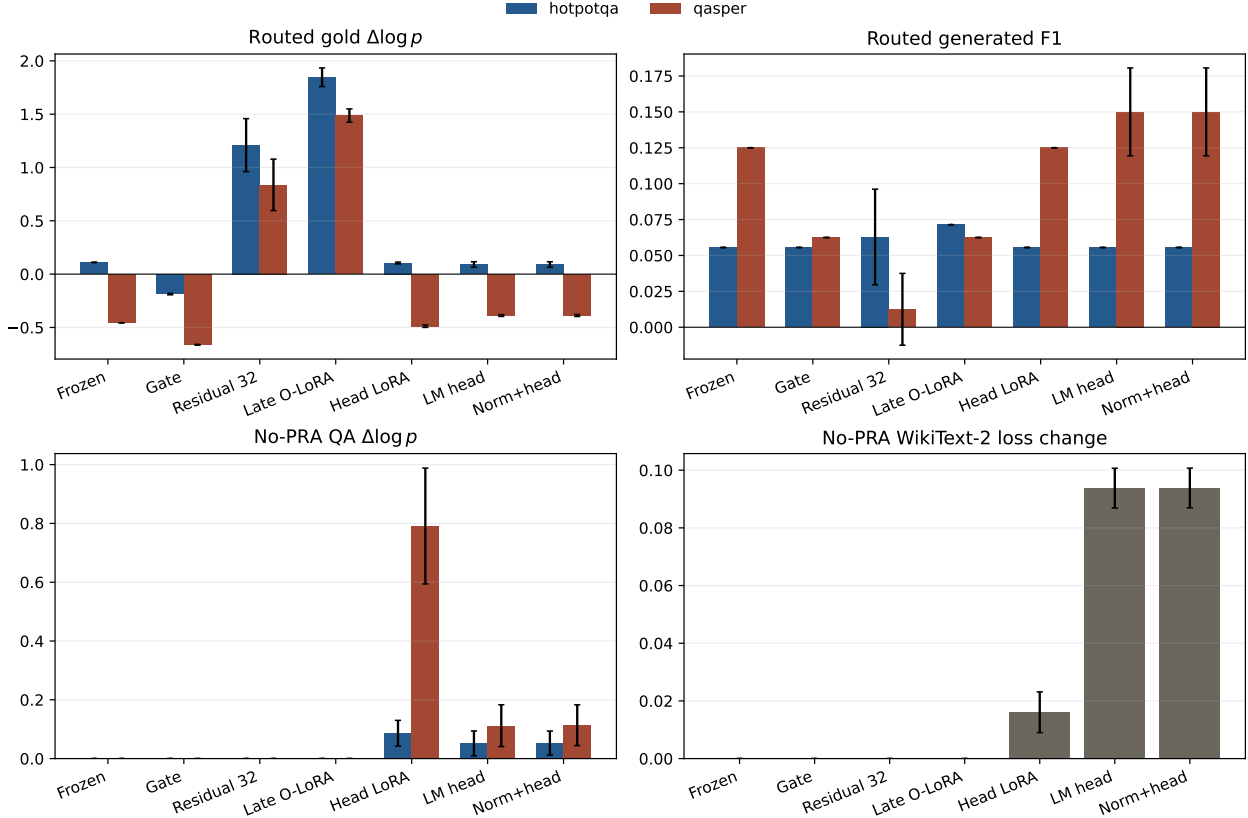


Figure 10: The complete adaptation sequence. Conditional residual and late output-LoRA improve the routed-minus-no-memory likelihood effect. Global readout controls do not, alter no-PRA QA scores, and increase WikiText-2 loss. QASPER is shown for sensitivity but is not used for the stop decision because its direct-text control fails.

The sequence separates answer-token calibration from memory use. Head LoRA changes no-PRA HotpotQA likelihood by $+0.086 \pm .044$ nats/token yet leaves the routed memory increment at $+0.104 \pm .009$, no better than frozen PRA. The full head changes the no-PRA score by $+0.052 \pm .042$ while reducing the memory increment to $+0.090 \pm .026$. Adding final normalization changes neither result. On WikiText-2, head LoRA raises loss by $+0.0161 \pm .0071$ and both full-head variants by about $+0.0938 \pm .0069$, increasing perplexity from 48.48 to 49.27 and 53.25. Global adaptation therefore spends 12.4 or 1,583 times the parameters of late output-LoRA, weakens exact retrofit, and does not teach the decoder to use PRA memory more effectively.

The stop decision is consequently narrow. The scalar gate is insufficient. Residual-32 and late output-LoRA demonstrate memory-specific likelihood calibration, with rank-8 late output-LoRA the most parameter-efficient measured choice. None produces reliable generation: HotpotQA EM remains zero and the best F1 is .071. The global control rules out a simple LM-head bottleneck on this cohort, but it does not solve decoding. Following the paper boundary, we stop rather than widen LoRA to Q, MLP, or the whole model. A larger identity-disjoint generation study is the next unit of evidence.

11 Product and Systems Boundary

The validated path is packaged as `pra-hf`, with a stable Python API, CLI, serializable configuration, fraction-based selection, and Hugging Face-style router artifacts. Version `0.2.0rc1` builds as a wheel and is pinned by tag `paper2-hf-rc1`. A caller adds a reference and generates through the same model object:

```
from pra_hf import PRAConfig, PRAForCausalLM

model = PRAForCausalLM.from_pretrained(
    BASE_MODEL, routing_adapter=ROUTER,
    pra_config=PRAConfig(selected_fraction=0.20))
ref = model.add_reference_file("paper.txt")
result = model.generate(question, return_details=True)
model.remove_reference(ref)
```

The API also supports in-memory text, explicit URI/text pairs, chat formatting, `#_head` rollover, cache clearing, and per-request statistics. A router artifact contains projection weights, configuration, and a model card; it pins the base model revision, family, routing layer and representation, parent size, dataset splits, seeds, metrics, PRA version, and source revision without republishing base-model weights. Qwen’s router has 262,144 parameters (0.044% of 596,049,920); SmolLM2’s has 147,456 (0.110% of 134,515,008). Both store one 128-dimensional FP32 vector, 512 bytes, per parent.

Memory accounting uses three denominators. *Routing-index bytes* are the compact vectors searched once per request. *Resident detail bytes* are all layer-native K/V retained on CPU. *Materialized bytes* are the selected K/V transferred for each consuming layer. A 20% requested parent fraction need not become 20% active K/V: whole-parent rounding and the hard token cap may reduce it. Conversely, replaying one selected identity through eight layers incurs eight layer-specific payloads even though semantic routing happened once.

Table 18: **The same product path runs on Qwen3-0.6B and SmolLM2-135M with bounded materialization.** Four-example CUDA means on a GTX 950M are a systems demonstration, not a throughput or QA benchmark. Peak GPU allocation includes the full model.

Family	Evidence	F1	Requested	Materialized	Route ms	Peak GiB
Qwen3-0.6B	0.667	0.033	20.4%	6.61%	6.40	1.126
SmolLM2-135M	0.750	0.028	21.0%	6.25%	32.74	0.261

Qwen averages a 45.9 KiB routing index, 91.3 MiB of resident eight-layer detail K/V, and 4.0 MiB transferred per request across the eight consuming layers. SmolLM2 averages 48.8 KiB, 18.2 MiB, and 0.75 MiB, respectively. Mean wall time for 16 routed output tokens is 2.15 seconds for Qwen and 1.85 seconds for SmolLM2, with no native-limit violations. These values describe an eager Python implementation with exact Torch search. They are not evidence of serving speedup: there is no matched dense long-context baseline, fused kernel, asynchronous transfer, or concurrency study.

The public implementation makes the tested boundary reusable. It preserves native GQA storage rather than permanently expanding K/V heads, reports both requested and actual materialization, and permits exact disabled-path testing for a new family. The release includes examples,

router save/load artifacts, package and CLI tests, and 275 passing repository tests at the pinned revision. The thin Llama adapter resolves installed rotary and eager-attention functions while the shared core retains publication, routing, budgeting, and replay. Full code-level guidance appears in Appendix B.

12 Limitations and Next Experiments

The experiments establish mechanisms on modest cohorts, not benchmark accuracy. The primary representation study uses 16 matched examples, the learned router has 32 held-out test identities, and the oracle depth study uses four examples per dataset. Five seeds measure optimization variation on fixed identities; they do not replace a larger test set. The 1,024-token WikiText-2 control can detect drift but is not a broad language-capability evaluation.

Evidence labels and answer behavior measure different things. Any-evidence recall can be high while a multi-hop example still lacks another required passage. Evidence-identity recall is stricter, but it still assumes that annotations identify the state this checkpoint needs. The positive HotpotQA direct-text control makes its causal comparison interpretable. QASPER direct text lowers the target likelihood on the selected causal subset, so that subset cannot grade PRA transport in isolation. Reliable decoded QA gains remain unestablished on both datasets.

The router is domain-dependent and memory exposure is depth-sensitive. Cross-domain any-evidence recall@3 is 0.213 in both directions. Additional gist resolution, centered-RoPE features, query pooling, width, and negative-mining variants do not remove this limitation. All-layer native-K/V injection is also harmful despite using more memory. A larger study should separate calibration, model selection, and test identities; report any-, identity-, and all-evidence recall against both selected-parent and exact-K/V fractions; and compare no-memory, routed, oracle, and direct-text conditions at a selected late band.

The systems result is bounded execution, not serving speed. Detail K/V remains CPU-resident, search is eager and exact, and selected rows are padded. Fused attention, approximate indexing, asynchronous transfer, batching, concurrency, and a matched dense long-context baseline remain unmeasured. Meta Llama 3.2-1B is also pending because checkpoint access returned HTTP 401; SmolLM2-135M is reported only as a measured Llama-family control.

The next Paper 2 experiment is a larger checkpoint-solvable generation cohort at the late-band operating point. Recursive multi-hop gathering belongs to Paper 2.5. Multi-gist strategy, semantic positional distance, and large-scale vLLM serving are separate research questions. Global LM-head tuning is not the next adaptation step: it fails to improve the memory-specific increment, changes no-PRA behavior, and raises WikiText-2 loss.

13 Related Work

PRA differs from text retrieval-augmented generation, which retrieves text and lets the model encode it in the current prompt [4]. It retrieves layer-native K/V after a separate bounded encoding path. This makes PRA complementary to RAG: a text retriever may choose documents before PRA routes chunks inside cached model state. Sparse-attention systems such as Longformer and BigBird constrain token-token connectivity inside a forward pass [1, 7]; PRA instead materializes selected state from an external addressable cache. KV-cache systems emphasize serving capacity and token retention policies [3, 8, 5]. Paper 2 asks whether selection can be inserted beneath an existing pretrained family without changing its disabled behavior.

14 Conclusion

PRA-HF retrofits pretrained Hugging Face decoders with external native-K/V memory while preserving their ordinary behavior exactly when disabled. References are encoded in model-safe blocks, detail K/V may remain off the active path, and a compact index selects whole chunks for bounded native attention. The shared core works through thin Qwen and Llama-family adapters without checkpoint conversion.

The architectural result is to route semantic state and attend native positional state. Hidden-state gists avoid the positional bias of post-RoPE key means, while selection still returns the host layer’s exact post-RoPE keys and native values. A 0.044%-of-model Qwen router reaches any-evidence recall 0.831 at 10% and 0.956 at 20% selected parents. Matched QASPER results on SmolLM2 show that the shared interface, rather than identical retrieval quality, is portable.

Selected memory measurably affects pretrained predictions. HotpotQA oracle replay gains 3.641 nats per gold token over the last 14 layers but loses 0.766 over all 28, establishing a bounded depth regime rather than a monotone layer-count effect. The implicit-head probe strengthens the separation among stages: evidence ranks first and gold likelihood rises by 5.469 nats per token, yet the greedy answer remains wrong. A 98,304-parameter PRA-conditional update improves routed likelihood further, whereas much larger global readout controls do not improve the memory-specific increment and disturb ordinary language modeling.

The released `pra_hf` package, artifacts, metrics, tests, and reimplementations guide make this exact portable retrofit independently usable. PRA-HF therefore establishes sparse semantic selection and depth-dependent causal use of large native-K/V memory; reliable decoded task gains remain the principal open boundary.

A Routing Ablations

This appendix records the secondary interventions that shaped the final design. They are useful negative results: each changes one plausible source of routing error while leaving native detail K/V unchanged.

Table 19: Parameter-free diagnostics with 32-token parents. Cohorts differ as described in the artifact metadata, so rows summarize directions rather than a single paired leaderboard.

Intervention	Sparse result	Cost or control	Interpretation
Hidden segment means, $G = 1/2/4/8$	Any R@3 0.391/0.313/0.266/0.281	Index overhead 1.57/3.14/6.28/12.56%	Finer untrained means do not improve sparse ranking
Token-level hidden maximum	Any R@3 0.375 versus mean 0.438	Index overhead 50% versus 1.57%	Resolution helps some Hotpot ranks but hurts QASPER
Centered-RoPE, $G = 8$	Any R@3 0.250	Token-QK order correlation 0.874	Coordinate repair does not create evidence semantics
Question decay, strongest tested	Any R@3/MRR 0.250/0.279	Last state 0.469/0.413	More query tokens are not automatically a better query
Width 256 and harder negatives	No validation-selected gain	Twice the 128-wide index at width 256	Data and objective generalization dominate raw capacity

The chunk-size sweep supplies a related caution. At 128-token chunks and $k = 8$, recall reaches 0.688 but the selected fraction is 41.3%; at $k = 16$, 71.5% of parents are selected. Coarser chunks can improve apparent fixed- k recall by making each choice cover more source text. That is not a sparse win, which is why the main paper reports selected fractions and physical K/V.

Independent hidden-state confirmation over 64 evaluations gives any/all-evidence recall of 0.391/0.063 at $k = 3$, 0.578/0.203 at $k = 8$, and 0.797/0.500 at $k = 16$. The selected-parent fractions are 0.045, 0.119, and 0.238, while the hard materialization budget holds the latter two near 0.059 physically active K/V. This illustrates all three distinctions at once: any versus all evidence, selection versus materialization, and fixed rank versus selected fraction.

B Code-Level Adapter Reimplementation Guide

This appendix gives the minimum code structure needed to reproduce the integration without copying the Qwen implementation. Source references use physical line numbers at repository commit `db50846`. This checkpoint makes attention-input hidden-state routing the explicit default and exposes the fixed-selection materialization boundary. The line references are deliberately revision-pinned because upstream Transformers APIs and local diagnostics will move over time. The multi-layer identity mapping, experiments, tests, and complete artifacts are pinned separately at commit `8f6ef41`; all multi-layer line references below refer to that revision.

B.1 Tensor and ownership contract

A new family should add projection, position, cache, mask, kernel, and output conventions in its adapter, while leaving selection and memory policy in the shared core.

The canonical tensor contract is

$$Q \in \mathbb{R}^{B \times H_q \times T_q \times d_h}, \quad K, V \in \mathbb{R}^{B \times H_{kv} \times T_k \times d_h}. \quad (13)$$

Reference entries use $B = 1$ and keep H_{kv} , including for GQA or MQA. They are not expanded to H_q in persistent storage. The controlled replay implementation shows the only required expansion point in `memory_batching.py`, lines 216–290: after memory and local K/V are joined, native K/V heads may be repeated for the attention computation. Qwen’s installed eager kernel performs the corresponding operation in the production adapter.

B.2 Minimal forward-path skeleton

The following pseudocode is the adapter’s essential control flow. It is intentionally family-neutral; “native” operations must call the installed model implementation rather than reimplementing its mathematics.

```

01 forward(hidden, native_positions, native_mask, past):
02     if capture_is_armed:
03         q_pre, k_pre, v = native_project_and_norm(hidden)
04         q_post, k_post = native_position(q_pre, k_pre)
05         save_transient_routing_features(hidden, q_pre, q_post, k_pre)
06         save_post_position_detail_kv(k_post, v)
07
08     if memory_is_disabled or reference_cache_is_empty:
09         return original_attention(hidden, native_positions,
10                                 native_mask, past)
11
12     q_pre, k_pre, v = native_project_and_norm(hidden)
13     q_post, k_post = native_position(q_pre, k_pre)

```

```

14     if past is not None:
15         k_post, v = past.update(k_post, v, layer_id, native_kwargs)
16         route_q = matched_query(hidden[:, -1, :], q_pre, q_post)
17         memory = shared_core.prepare_memory(q_post, route_q,
18                                             direct_tokens=k_post.shape[2])
19         if memory.is_empty:
20             return native_attention_and_output(q_post, k_post, v, native_mask)
21         k, v, mask = shared_core.combine_local_and_memory_kv(
22             k_post, v, memory, native_mask, query_tokens=q_post.shape[2])
23         return native_attention_and_output(q_post, k, v, mask)

```

Lines 8–10 are a correctness boundary, not an optimization. The Qwen implementation delegates to the untouched module at `hf/qwen.py`, lines 190–201, which makes disabled behavior auditable and bit-exact. In the active path, projection and RoPE occur at lines 204–207 and `Cache.update` occurs once at lines 208–215. Lines 216–221 choose the matched routing query without altering the post-RoPE attention query. If routing returns no materialized memory, lines 224–234 call the native kernel directly. Delegating at that point would execute a second cache update and duplicate generation history.

Memory is composed only after the native cache update. The shared operation at `core.py`, lines 394–436, pads variable memory rows, prepends memory K/V, creates an all-visible additive mask for valid memory positions, masks padding, and concatenates the unchanged local mask. The adapter then invokes Qwen’s eager function and one pretrained `o_proj` at `hf/qwen.py`, lines 92–110 and 246–258. Thus local and memory positions share one softmax and one output projection.

B.3 Optional learned routing metric

The learned adapter is a replaceable routing component, not part of native attention. For an asymmetric linear metric,

$$r_q = \text{norm}(W_q h_q), \quad r_c = \text{norm}(W_c g_c), \quad s(q, c) = r_q^\top r_c. \quad (14)$$

A shared adapter aliases $W_q = W_c$. An independent implementation needs only the following boundary, revision-pinned at commit `a657bc7`:

```

01 # Reference publication, after ordinary hidden-state gist pooling
02 routing_gist = adapter.project_memory(routing_gist)
03 cache.store(routing_gist, native_post_rope_k, native_v)
04
05 # Live routing; native attention continues to use q_post
06 information_need = aggregate_query_states(h_in, strategy)
07 routing_query = adapter.project_query(information_need)
08 selected_ids = cache.search(routing_query)
09 k_mem, v_mem = cache.materialize_native_kv(selected_ids)
10 native_attention(q_post, concat(k_mem, k_local),
11                  concat(v_mem, v_local))

```

The projection implementation is `hf/routing_adapter.py`, lines 10–74; checkpoint loading and freezing are lines 77–94. Memory projection occurs at `hf/injection.py`, lines 238–246, and live query projection at `hf/qwen.py`, lines 188–202. Cast hidden features to the adapter parameter

dtype at this boundary; the checkpoint is FP32 while Qwen emits FP16 states. Do not cast, project, or recompute native detail K/V. Their exact-equality invariant is the test that prevents a routing retrofit from silently becoming an attention rewrite.

B.4 Oracle selection and causal diagnostics

An oracle must replace only identity selection. Adding a separate K/V injection path would confound semantic correctness with different budgeting, positions, masks, or attention code. The operational handle activates a layer set and assigns optional row-local identities in `hf/injection.py`, lines 69–89. The Qwen adapter chooses ordinary routing or the fixed set at `hf/qwen.py`, lines 273–286; both enter `core.py::prepare_selected_memory`, lines 337–388. The latter still applies the trigger, deduplication, whole-chunk budget, overlap policy, residency transfer, and rectangular packing. The fixed-set setter and its ownership contract are in `hf/adapter.base.py`, lines 71–83.

The experiment constructs oracle identities by intersecting annotation token spans with parent logical spans in `run_oracle_memory_use.py`, lines 107–134. Lines 194–281 append the gold continuation, gather its autoregressive token probabilities, retain opt-in eager-attention weights, and use temporary decoder-layer hooks for answer-position hidden states. Attention capture is disabled immediately afterward and is never part of ordinary inference. Conditions and paired deltas are assembled at lines 310–419. A reimplementer should assert three invariants per example: requested oracle IDs equal retained IDs, budget rejection is zero, and the largest native operation stays below the declared bound.

B.5 Route-once multi-layer consumption

Multi-layer PRA shares an identity decision, not an attention tensor. An implementation should make the distinction visible in its types:

```
01 selected = route_once(query_at_layer_27)          # parent IDs + scores
02 for layer in consumption_layers:
03     layer_hits = []
04     for hit in selected:
05         chunk = cache[hit.uri].layer[layer].by_id[hit.chunk_id]
06         assert chunk.kv.was_projected_by == layer
07         layer_hits.append(hit.with_payload(chunk.kv))
08     adapter[layer].set_fixed_selection(layer_hits)
09 model(question)                                  # no further semantic search
```

The operational API activates an arbitrary layer set at `hf/injection.py`, lines 70–93. Identity-to-payload resolution is lines 97–145: for each requested layer it looks up the stable `chunk_id` in that layer’s `LayerReferenceMemory`, replaces the payload, and records the source routing layer. A missing layer or chunk is an error rather than a fallback to another layer’s K/V. The Qwen adapter’s fixed-selection branch at `hf/qwen.py`, lines 273–289 then calls the same budgeting, transfer, mask, and attention path as ordinary routing.

The experiment captures the last-layer information-need state and performs one complete ranking at `run_multilayer_pra.py`, lines 161–198. Oracle and routed identities are mapped at lines 277–325; the remainder of the function records per-layer attention, hidden-state changes, materialized tokens, transfers, and latency. The invariant test at `tests/test_hf_integration.py`, lines 548–586 asserts equal parent IDs but different target-layer objects, tensor pointers, and values while

preserving native $[B, H_{kv}, T, d_h]$ shape. This catches the most dangerous superficially working implementation: copying $K^{(27)}, V^{(27)}$ into every consuming layer.

B.6 Publishing a reference

Reference construction is a bounded prefill, not a second attention mechanism. A portable implementation follows this recipe:

```

01  disable_PRA_memory_during_reference_encoding()
02  for [block_start: block_end] in bounded_encoding_blocks(source):
03      position_ids = logical_source_positions(block_start, block_end)
04      arm_post_position_kv_capture(position_ids)
05      model(block_ids, position_ids=position_ids, use_cache=False)
06      captured = consume_native_kv_for_each_PRA_layer()
07      for routing_window in overlapping_windows(captured):
08          gist = pool(routing_window.attention_input_hidden_state)
09          publish(uri, logical_offsets, post_rope_k, native_v, gist)
10          discard_token_level_hidden_state()
11  restore_previous_memory_state()

```

The concrete loop is `hf/injection.py`, lines 235–354. Encoding block size limits one native model invocation; routing chunk size and overlap independently determine index granularity (lines 266–292). Each chunk slices the captured post-RoPE K/V, chooses transient routing features through `_routing_points` (lines 172–233), computes one or more compact gists, and records logical source offsets (lines 293–348). Keeping those two scales separate is important: changing index granularity must not silently change how the pretrained model encoded the source.

The contiguous multi-gist extension keeps that ownership boundary explicit. For a requested count G , split the routing feature rows into balanced source-order ranges, mean each range, score all resulting $[G, D]$ rows, reduce to one parent score, and preserve the winning local index for diagnostics. Top- k must then operate on parent identities before budgeting; otherwise several winning gists could duplicate one parent K/V payload. The reference implementation is revision-pinned at 0205df8: pooling is in `gists/segment_mean.py`, lines 11–56, publication remains `hf/injection.py`, lines 163–190, and packed scoring plus max reduction remains `memory.py`, lines 487–593.

The same operation implements displaced long-prompt history. At `hf/injection.py`, lines 357–370, the prefix is published under the reserved `#_head` URI and the direct tail retains continued logical position IDs. Every reference-encoding call and every direct call must satisfy the model-safe native bound even when the logical source is longer.

B.7 Porting another decoder family

A practical port begins by locating seven hooks in the installed family: the attention module inside each decoder layer; Q/K/V projections; any Q/K normalization; positional encoding; generation-cache update and its keyword arguments; native mask convention; and the eager attention plus output projection. Implement the six abstract methods declared at `hf/adapters_base.py`, lines 85–106, then add a narrow type-selection branch analogous to `inject_pra` at `hf/injection.py`, lines 379–428. No checkpoint conversion or parameter copy should be needed.

The recommended validation order is:

1. With memory disabled, require exact logits, hidden states, greedy tokens, and generation- cache lengths against an unwrapped copy (test lines 55–72).
2. Capture a short reference and verify post-position state, logical offsets, device residency, and H_{kv} rather than H_q storage (lines 74–93).
3. Verify matched pre/post capture, representation switching, unchanged post-RoPE detail K/V, GQA grouping, serialization, and device parity (lines 109–214).
4. Supply one selected identity through router and oracle traces; require identical native K/V and attention output (lines 241–280).
5. Compare GQA replay with explicit head repetition on a synthetic tensor (lines 216–239).
6. Exercise an over-limit logical prompt and verify bounded native operations plus continued position IDs (lines 282–313).
7. Generate with active memory and assert that direct cache length grows by exactly one per decoded token (lines 315–330).

Four mistakes deserve explicit tests. Applying position encoding again to stored post-position keys changes their basis. Permanently expanding GQA K/V multiplies memory without adding information. Replacing the family mask with a generic causal mask can alter padding or sliding-window behavior. Finally, calling the wrapped module after manually updating its cache appends the same local token twice. Passing transport and parity tests establishes a faithful adapter; it does not establish that the router selects causally useful evidence, which requires the separate quality controls reported in the main paper.

B.8 Llama port and product API

The release implementation is pinned at commit 809694b. The common abstract family contract remains in `adapter_base.py`, lines 32–132. Qwen’s implementation resolves its installed helper functions through an overridable method at `qwen.py`, lines 38–77. The Llama module at `llama.py`, lines 8–48, subclasses that shared projection/capture/control path, replaces the symbol resolver, and invokes Llama’s native eager function without the Qwen sliding-window argument. This is inheritance of a family-neutral tensor contract, not reuse of Qwen parameters. Injection selects Qwen or Llama from the concrete attention module at `injection.py`, lines 380–430.

An independent family port should make the same decision only after comparing installed forward signatures and testing every shared assumption. For Llama those assumptions are: Q/K/V linear outputs reshape to $[B, H, T, d_h]$; RoPE receives the same cosine/sine tuple; cache update accepts the layer index, sine, cosine, and cache position; the eager kernel repeats native K/V heads internally; and output returns token-major states before one native output projection. If any family differs, override that specific abstract method rather than adding a family branch to the router, cache, or budgeter. Tests in `test_hf_llama_integration.py`, lines 51–106, cover exact disabled behavior, native-head reference payloads, active memory, and rollover.

The public layer composes rather than replaces those internals. `pra_hf/config.py`, lines 14–127, validates stable product names, resolves negative layer indices, and translates them to the core configuration. `pra_hf/router.py`, lines 14–110, defines the versioned weights/metadata/model-card artifact and legacy-checkpoint conversion. The high-level model in `pra_hf/model.py`, lines 42–358, owns tokenizer/model loading, router compatibility, reference lifecycle, fraction selection,

route-once identity replay, generation, chat, and memory economics. In particular, lines 192–258 flatten complete reference-grouped rankings, apply a global fraction or top- k cutoff, reconstruct typed identities, and map those IDs to each consuming layer’s native payload. The CLI in `pra_hf/cli.py`, lines 22–145, is a thin caller of the same API; it does not maintain a second execution path. Product tests at `test_pra_hf_api.py`, lines 66–132, pin config serialization and precedence, router save/load and freezing, add/remove/clear, fraction routing, generation, statistics, and the rule that a router cannot change after gists have been indexed.

B.9 Memory calibration path

The calibration code preserves the same ownership boundary. The registered scalar bank in `hf/memory_gate.py`, lines 19–94, supports fixed, shared, and per-layer values. The residual bank in `hf/residual_adapter.py`, lines 10–111, lazily constructs only the requested bottleneck and enables gradients only for that width. Its zero output projection is what makes Equation 11 initially equal frozen PRA rather than a random perturbation.

The Qwen/Llama forward path computes y^{mem} first. Only when a trainable gate or residual adapter is active does `hf/qwen.py`, lines 358–405, invoke local attention a second time, form d_l , apply the gate and adapter in FP32, and cast the result back to the model dtype. The ordinary fixed-PRA path therefore does not pay for the counterfactual attention. Registration and public configuration live in `hf/injection.py`, lines 94–106 and 444–463; adapters hold non-owning references so the model registers one parameter bank rather than duplicating it under every attention wrapper.

The reproducible protocol is in `experiments/paper2_hf/qa/run_memory_gate.py`: lines 158–234 freeze and optimize the selected owner, lines 236–331 evaluate exact no-memory, oracle, and routed conditions, and lines 332–461 aggregate and plot seed-level measurements. Unit tests at `tests/test_hf_integration.py`, lines 83–118 and 577–710, pin parameter ownership, zero-gate behavior, frozen-PRA initialization, adapter-only gradients, and exact disabled-PRA bypass.

B.10 Conditional late-band LoRA path

The LoRA implementation is deliberately separate from generic parameter-efficient fine-tuning. `hf/late_band_lora.py`, lines 10–50, defines one rectangular low-rank delta that accepts the pre-output-projection tensor $[B, T, D_a]$ and returns only a correction $[B, T, D]$. Lines 53–149 own a lazy per-layer bank, activate one rank/configuration at a time, and expose only that configuration’s trainable factors. This supports Qwen3’s 2048 $\rightarrow$ 1024 projection as well as Llama’s square projection without family-specific dimensions in the LoRA class.

Registration occurs once in `hf/injection.py`, lines 467–482. Each attention wrapper holds a non-owning reference to that bank, avoiding duplicate parameter registration. Public rank switching and parameter selection are at lines 110–128. In the active Qwen/Llama path, `hf/qwen.py`, lines 348–374, first performs native attention over local plus selected K/V, flattens heads to $[B, T, D_a]$, applies the frozen W_O , and then adds Equation 12 in FP32. Reaching those lines already implies that memory exists. The earlier no-memory return and the disabled wrapper therefore never execute LoRA, which is the structural reason the no-PRA control is bit-exact rather than merely regularized toward parity.

The experiment entry point `run_late_band_lora.py`, lines 73–192, configures one exclusive calibration owner and trains it while evaluating a no-PRA control on every update. Lines 194–293 evaluate all four memory conditions; lines 295–353 plot the aggregate; and lines 355–547 run the fixed five-seed cohort and write raw, seed-level, and aggregate artifacts. Tests

in `test_hf_integration.py`, lines 122–152 and 739–790, pin validation, rectangular shape handling, zero initialization, parameter ownership, active gradients, and exact Qwen bypass. `test_hf_llama_integration.py`, lines 124–153, exercises the same ownership and bypass contract through the Llama-family wrapper.

B.11 Global readout control

The LM-head experiment is intentionally outside the product adapter. The class `LMHeadLoRA` in `run_lm_head_control.py`, lines 51–89, holds a non-owning reference to the frozen tied head and registers only rectangular low-rank factors. The zero output factor makes its initial logits exact. Lines 92–124 restore the native head and final normalization before every variant; full-head controls clone the output matrix so optimization cannot mutate the tied input embeddings.

Lines 126–171 build fixed cached WikiText blocks and compute ordinary no-PRA causal loss. Lines 173–300 train oracle-memory examples and the frozen-logit KL control with sequential backward passes, which preserves the summed objective while bounding activation memory. Lines 302–405 evaluate no memory, routed memory, oracle memory, and direct text against both the current and frozen no-memory baselines. Lines 407–668 aggregate the language and QA controls and join them to the earlier gate, residual, and late-LoRA artifacts; lines 670–891 execute the common cohort. Tests in `test_hf_lm_head_control.py`, lines 9–37, pin zero-initialized LoRA equality, factor-only ownership, and exact-but-untied full-head cloning.

B.12 Revision-pinned source map

Table 20 connects the preceding control flow to the implementation.

Table 20: Source map for an independent adapter implementation.

Responsibility	File and physical lines
Abstract family boundary	<code>src/pras_torch/hf/adapter_base.py</code> , 16–106
Qwen projections, RoPE, native kernel, and output	<code>src/pras_torch/hf/qwen.py</code> , 53–110
Llama native-symbol binding and eager invocation	<code>src/pras_torch/hf/llama.py</code> , 8–48
Matched routing/detail capture	<code>src/pras_torch/hf/qwen.py</code> , 109–148; <code>src/pras_torch/hf/injection.py</code> , 80–210
Disabled and active runtime paths	<code>src/pras_torch/hf/qwen.py</code> , 171–275
Routing query and GQA reduction	<code>src/pras_torch/core.py</code> , 92–121
Budget, deduplication, transfer, and materialization	<code>src/pras_torch/core.py</code> , 155–392
Fixed-selection identity/payload boundary	<code>src/pras_torch/core.py</code> , 335–392
Route-once layer-native identity mapping	<code>src/pras_torch/hf/injection.py</code> , 70–145
K/V and additive-mask composition	<code>src/pras_torch/core.py</code> , 394–436
Reference publication and selected-layer injection	<code>src/pras_torch/hf/injection.py</code> , 235–428
Depth schedules and causal protocol	<code>experiments/paper2_hf/qa/run_multilayer_pra.py</code> , 62–964
Memory gate and residual calibration	<code>src/pras_torch/hf/memory_gate.py</code> , 19–94; <code>src/pras_torch/hf/residual_adapter.py</code> , 10–111; <code>src/pras_torch/hf/qwen.py</code> , 358–405
Five-seed calibration protocol	<code>experiments/paper2_hf/qa/run_memory_gate.py</code> , 158–461
Conditional output LoRA	<code>src/pras_torch/hf/late_band_lora.py</code> , 10–149; <code>src/pras_torch/hf/qwen.py</code> , 348–374
Five-seed LoRA protocol	<code>experiments/paper2_hf/qa/run_late_band_lora.py</code> , 73–547
Global readout and ordinary-language control	<code>experiments/paper2_hf/qa/run_lm_head_control.py</code> , 51–891; <code>tests/test_hf_lm_head_control.py</code> , 9–37
Parity, routing, GQA, head, cache, and selection gates	<code>tests/test_hf_integration.py</code> , 80–790
Stable config, router artifact, and public API	<code>src/pras_hf/config.py</code> , 14–127; <code>src/pras_hf/router.py</code> , 14–110; <code>src/pras_hf/model.py</code> , 42–358
Llama and product-facing gates	<code>tests/test_hf_llama_integration.py</code> , 51–153; <code>tests/test_pra_hf_api.py</code> , 66–132

References

- [1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [2] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021. Preprint: <https://arxiv.org/abs/2105.03011>.
- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

- [5] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
- [6] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018. Preprint: <https://arxiv.org/abs/1809.09600>.
- [7] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
- [8] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.